

Rapport de TP4 : Neo4j & Langage Cypher

Bases de données non relationnelles — SGBD NoSQL orientés Graphes

Étudiant : Makhtar

Année Académique : 2025 - 2026

Sujet : Réseau Social & Recommandation

Technologie : Neo4j (v5) via Docker

Introduction

Ce travail pratique est consacré à la modélisation et à l'interrogation de bases de données de graphes à l'aide de Neo4j et de son langage de requête déclaratif, Cypher. L'objectif est de mettre en œuvre un mini réseau social connectant des personnes caractérisées par des relations de suivi réciproques ou unilatérales, ainsi que par leurs centres d'intérêt.

Partie 1 — Modélisation et Chargement du Graphe

Le graphe est constitué de deux types de nœuds principaux :

Personne : caractérisé par les propriétés `nom` (clé unique), `age`, et `ville`.

Interet : caractérisé par le nom du centre d'intérêt (Sport, Musique, Tech, Voyage).

Les interactions entre ces entités sont modélisées par les relations typées suivantes :

(:Personne) -[:SUIT]->(:Personne) (réseau orienté représenté par des abonnements).

(:Personne) -[:AIME]->(:Interet) (appartenance ou goût pour un centre d'intérêt).



Script de chargement et d'initialisation

```

// Nettoyage complet
MATCH (n) DETACH DELETE n;

// Création des centres d'intérêt
CREATE (:Interet {nom: "Sport"});
CREATE (:Interet {nom: "Musique"});
CREATE (:Interet {nom: "Tech"});
CREATE (:Interet {nom: "Voyage"});

// Création des 8 personnes
CREATE (:Personne {nom: "Alice", age: 28, ville: "Paris"});
CREATE (:Personne {nom: "Bob", age: 32, ville: "Lyon"});
CREATE (:Personne {nom: "Charlie", age: 25, ville: "Paris"});
CREATE (:Personne {nom: "Diana", age: 29, ville: "Marseille"});
CREATE (:Personne {nom: "Eve", age: 35, ville: "Lille"});
CREATE (:Personne {nom: "Franck", age: 40, ville: "Lyon"});
CREATE (:Personne {nom: "Grace", age: 22, ville: "Paris"});
CREATE (:Personne {nom: "Hugo", age: 31, ville: "Bordeaux"});

// Liens de suivi (relations SUIT)
MATCH (alice:Personne {nom:"Alice"}),(bob:Personne {nom:"Bob"}),(charlie:Personne {nom:"Charlie"}),(diana:Personne {nom:"Diana"});
CREATE (alice)-[:SUIT]->(bob),(alice)-[:SUIT]->(charlie),(alice)-[:SUIT]->(diana),(bob)-[:SUIT]->(alice),(bob)-[:SUIT]->(charlie),(bob)-[:SUIT]->(diana),(charlie)-[:SUIT]->(alice),(charlie)-[:SUIT]->(bob),(charlie)-[:SUIT]->(diana),(diana)-[:SUIT]->(alice),(diana)-[:SUIT]->(bob),(diana)-[:SUIT]->(charlie);

// Liens d'intérêt (relations AIME)
MATCH (alice:Personne {nom:"Alice"}),(bob:Personne {nom:"Bob"}),(charlie:Personne {nom:"Charlie"}),(diana:Personne {nom:"Diana"});
CREATE (alice)-[:AIME]->(Sport),(alice)-[:AIME]->(Musique),(alice)-[:AIME]->(Tech),(alice)-[:AIME]->(Voyage),(bob)-[:AIME]->(Sport),(bob)-[:AIME]->(Musique),(bob)-[:AIME]->(Tech),(bob)-[:AIME]->(Voyage),(charlie)-[:AIME]->(Sport),(charlie)-[:AIME]->(Musique),(charlie)-[:AIME]->(Tech),(charlie)-[:AIME]->(Voyage),(diana)-[:AIME]->(Sport),(diana)-[:AIME]->(Musique),(diana)-[:AIME]->(Tech),(diana)-[:AIME]->(Voyage);
  
```

```
MATCH (alice:Personne {nom:"Alice"}),(bob:Personne {nom:"Bob"}),(charlie:Personne {nom:"Charlie"}),(diana:Personne {nom:"Diana"}),(diana:Personne {nom:"Diana"}),(diana:Personne {nom:"Diana"})
CREATE (alice)-[:AIME]->(tech),(alice)-[:AIME]->(voyage),(bob)-[:AIME]->(sport),(charlie)-[:AIME]->(musique),(diana)-[:AIME]->(sport),(diana)-[:AIME]->(musique)
```

Partie 2 — Requêtes Cypher et Analyses

Voici les requêtes exécutées sur notre jeu de données ainsi que les résultats retournés :

N°	Objectif & Requête Cypher	Explication technique	Résultats observés
1	Lister les personnes de Paris MATCH (p:Personne {ville: "Paris"}) RETURN p.nom;	Filtre de propriété simple sur le label :Personne.	"Alice" "Charlie" "Grace"
2	Suivis directs d'Alice MATCH (a:Personne {nom: "Alice"})-[:SUIT]->(p) RETURN p.nom;	Recherche des nœuds cibles d'une relation sortante :SUIT à partir d'un nœud de départ défini.	"Diana" "Charlie" "Bob"
3	Amis d'amis d'Alice (2 sauts) MATCH (a:Personne {nom: "Alice"})-[:SUIT]->()-[:SUIT]->(p) WHERE p.nom <> "Alice" RETURN DISTINCT p.nom;	Traversée à 2 niveaux. Le filtre exclut le retour à l'origine (Alice).	"Eve" "Franck" "Diana"
4	Intérêts communs avec Alice MATCH (a:Personne {nom: "Alice"})-[:AIME]->(i)<-[:AIME]-(p) RETURN DISTINCT p.nom, i.nom AS interet;	Recherche de motifs partagés (V-shape) entre deux personnes partageant le même nœud d'intérêt.	"Hugo", "Voyage" "Diana", "Voyage" "Grace", "Tech" "Charlie", "Tech"
5	Top 3 des plus suivis MATCH (p:Personne)<-[:SUIT]-() RETURN p.nom, count(*) AS suiveurs ORDER BY suiveurs DESC LIMIT 3;	Agrégation par comptage des relations de suivi entrantes, tri décroissant et	"Bob", 2 "Charlie", 2 "Alice", 2

N°	Objectif & Requête Cypher	Explication technique	Résultats observés
		restriction de résultat.	
6	Plus court chemin Alice → Eve <pre>MATCH path = shortestPath((a:Personne {nom:"Alice"})-[:SUIT*..10]-(e:Personne {nom:"Eve"})) RETURN path;</pre>	Utilisation de la fonction intégrée <code>shortestPath</code> pour trouver le chemin minimal en mode non orienté (jusqu'à 10 sauts maximum).	Alice → Grace → Eve (Longueur : 2 relations)
7	Suivis réciproques <pre>MATCH (a:Personne)-[:SUIT]->(b:Personne)-[:SUIT]->(a) WHERE elementId(a) < elementId(b) RETURN a.nom, b.nom;</pre>	Détection de relations réciproques. Le filtre <code>elementId</code> évite d'afficher le doublon inversé (B, A).	"Alice", "Bob"

Partie 3 — Algorithme de Recommandation d'amis

Le moteur de recommandation cible les "amis d'amis" qu'Alice ne suit pas encore, triés par le nombre d'amis communs.

Requête de Recommandation

```
MATCH (alice:Personne {nom: "Alice"})-[:SUIT]->(ami)-[:SUIT]->(suggestion)
WHERE NOT (alice)-[:SUIT]->(suggestion) AND alice <> suggestion
RETURN DISTINCT suggestion.nom, count(ami) AS amis_communs
ORDER BY amis_communs DESC
LIMIT 5;
```

Résultats de la requête

Rang	Profil recommandé	Nombre d'amis en commun	Identité des intermédiaires
1	Eve	2	Bob et Diana (suivis par Alice et qui suivent tous deux Eve)
2	Franck	1	Charlie (suivi par Alice et qui suit Franck)

Explication détaillée

- Exploration :** La clause `(alice)-[:SUIT]->(ami)-[:SUIT]->(suggestion)` parcourt le graphe à deux degrés (amis des personnes suivies par Alice).
- Filtre négatif :** `WHERE NOT (alice)-[:SUIT]->(suggestion)` garantit qu'Alice ne reçoit pas de suggestion pour des profils qu'elle suit déjà (comme Diana qui apparaissait initialement dans la Partie 2 Q3).
- Auto-exclusion :** Le prédicat `alice <> suggestion` évite de recommander Alice à elle-même si ses amis la suivent en retour.
- Classement :** L'agrégation `count(ami)` calcule le score de pertinence (nombre d'intermédiaires). Eve a un score de 2, la plaçant en tête des suggestions.

Partie 4 — Réponses aux Questions de Réflexion

1. Pourquoi ce type de requête (recommandation d'amis d'amis) est-il difficile en SQL pur ?

Dans un modèle de données relationnel classique (SQL), les relations "plusieurs-à-plusieurs" (comme le réseau de suivi) sont gérées à l'aide d'une table d'association (par exemple `suivis(id_suiveur, id_suivi)`).

Pour effectuer une recherche d'amis d'amis (2 sauts), nous devons faire une jointure de la table de suivi sur elle-même (un *self-join*). Si nous souhaitons chercher à 3, 4 ou 5 sauts, ou trouver le plus court chemin de longueur indéterminée, le nombre de jointures augmente à chaque étape. Chaque jointure force le SGBD relationnel à multiplier les tables sous forme de produits cartésiens intermédiaires et à scanner des index de clés primaires/étrangères. À grande échelle, le temps de calcul explose de façon exponentielle.

Neo4j élimine cette complexité grâce à l'approche "**Index-free Adjacency**" (Adjacence sans index). Chaque nœud possède en mémoire des adresses physiques directes pointant vers ses nœuds voisins. Parcourir une relation revient à suivre un pointeur mémoire, ce qui s'effectue en temps constant $O(1)$, indépendamment de la taille globale du graphe.

2. Quelle limite si le graphe contient 100 millions de nœuds ?

Avec 100 millions de nœuds et potentiellement plusieurs milliards de relations, les limitations et points de vigilance sont les suivants :

La taille de la RAM (PageCache) : Pour maintenir ses performances de pointeur mémoire, Neo4j doit conserver la structure topologique active du graphe en mémoire vive. Si la taille de la RAM est insuffisante, le système effectue des lectures/écritures sur le disque (I/O), ce qui effondre les performances de traversée.

Le phénomène de Super-nœuds (Dense Nodes) : Certains profils (ex: comptes de célébrités) possèdent des millions de relations entrantes ou sortantes. Lorsqu'un algorithme de traversée passe par un tel nœud, le nombre de chemins à explorer explose instantanément, entraînant des latences critiques (CPU/mémoire saturés).

Explosion combinatoire : Les requêtes effectuant des parcours larges sans limite stricte de profondeur (comme `[ :SUIT* ]`) peuvent toucher une part colossale du graphe. Il est indispensable de contraindre les recherches de chemin (par ex. `[ :SUIT*..3 ]`).

Adaptation de l'architecture : Pour de telles volumétries, les requêtes de recommandation en temps réel (OLTP) sont généralement complétées par du pré-calcul asynchrone (OLAP) via le module **GDS (Graph Data Science)** de Neo4j (ex: algorithmes de clustering, PageRank, ou calculs de similarité Cosine périodiques).